## Getting Started

You can use all Gemma 4 models with the latest version of Transformers. To get started, install the necessary dependencies in your environment:

`pip install -U transformers torch accelerate`

Once you have everything installed, you can proceed to load the model with the code below:

```
from transformers import AutoProcessor, Au

MODEL_ID = "google/gemma-4-26B-A4B-it"

# Load model
processor = AutoProcessor.from_pretrained(
model = AutoModelForMultimodalLM.from_pret
    MODEL_ID,
    dtype="auto",
    device_map="auto"
)
```

Once the model is loaded, you can start generating output:

```
# Prompt
messages = [
    {"role": "system", "content": "You are
    {"role": "user", "content": "Write a s
]

# Process input
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
    add_generation_prompt=True,
    enable_thinking=False
).to(model.device)
input_len = inputs["input_ids"].shape[-1]

# Generate output
outputs = model.generate(**inputs, max_new
response = processor.decode(outputs[0][inp

# Parse output
processor.parse_response(response, prefix=
```

To enable reasoning, set `enable_thinking=True` and the `parse_response` function will take care of parsing the thinking output.

Below, you will also find snippets for processing audio (E2B, E4B, 12B only), images, and video alongside text:

▼ Code for processing Audio

Make sure to install the following packages:

```
pip install -U transformers torch torchvision librosa accelerate
```

You can then load the model with the code below:

```
from transformers import AutoProcessor, Au

MODEL_ID = "google/gemma-4-26B-A4B-it"

# Load model
processor = AutoProcessor.from_pretrained(
model = AutoModelForMultimodalLM.from_pret
    MODEL_ID,
    dtype="auto",
    device_map="auto"
)
```

Once the model is loaded, you can start generating output by directly referencing the audio URL in the prompt:

```
# Prompt - add audio after text
messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "Tran
            {"type": "audio", "audio": "ht
        ]
    }
]

# Process input
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
    add_generation_prompt=True,
).to(model.device)
input_len = inputs["input_ids"].shape[-1]

# Generate output
outputs = model.generate(**inputs, max_new
response = processor.decode(outputs[0][inp

# Parse output
processor.parse_response(response, prefix=
```

▼ Code for processing Images

Make sure to install the following packages:

```
pip install -U transformers torch
torchvision accelerate
```

You can then load the model with the code below:

```
from transformers import AutoProcessor, Au

MODEL_ID = "google/gemma-4-26B-A4B-it"

# Load model
processor = AutoProcessor.from_pretrained(
model = AutoModelForMultimodalLM.from_pret
    MODEL_ID,
    dtype="auto",
    device_map="auto"
)
```

Once the model is loaded, you can start generating output by directly referencing the image URL in the prompt:

```
# Prompt - add image before text
messages = [
    {
        "role": "user", "content": [
            {"type": "image", "url": "https
            {"type": "text", "text": "What
        ]
    }
]

# Process input
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
    add_generation_prompt=True,
).to(model.device)
input_len = inputs["input_ids"].shape[-1]

# Generate output
outputs = model.generate(**inputs, max_new
response = processor.decode(outputs[0][inp

# Parse output
processor.parse_response(response, prefix=
```

▼ Code for processing Videos

Make sure to install the following packages:

```
pip install -U transformers torch
torchvision librosa accelerate
```

You can then load the model with the code below:

```
from transformers import AutoProcessor, Au

MODEL_ID = "google/gemma-4-26B-A4B-it"

# Load model
processor = AutoProcessor.from_pretrained(
model = AutoModelForMultimodalLM.from_pret
    MODEL_ID,
    dtype="auto",
    device_map="auto"
)
```

Once the model is loaded, you can start generating output by directly referencing the video URL in the prompt:

```
# Prompt - add video before text
messages = [
    {
        'role': 'user',
        'content': [
            {"type": "video", "video": "ht
            {'type': 'text', 'text': 'Desc
        ]
    }
]

# Process input
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
    add_generation_prompt=True,
).to(model.device)
input_len = inputs["input_ids"].shape[-1]

# Generate output
outputs = model.generate(**inputs, max_new
response = processor.decode(outputs[0][inp

# Parse output
processor.parse_response(response, prefix=
```

### Best Practices

For the best performance, use these configurations and best practices:

#### 1. Sampling Parameters

Use the following standardized sampling configuration across all use cases:

- `temperature=1.0`
- `top_p=0.95`
- `top_k=64`

### 2. Thinking Mode Configuration

Compared to Gemma 3, the models use standard `system`, `assistant`, and `user` roles. To properly manage the thinking process, use the following control tokens:

- **Trigger Thinking:** Thinking is enabled by including the `<|think|>` token at the start of the system prompt. To disable thinking, remove the token.
- **Standard Generation:** When thinking is enabled, the model will output its internal reasoning followed by the final answer using this structure: `<|channel>thought\n` **[Internal reasoning]** `<channel|>`
- **Disabled Thinking Behavior:** For all models except for the E2B and E4B variants, if thinking is disabled, the model will still generate the tags but with an empty thought block: `<|channel>thought\n<channel|>` **[Final answer]**

> Note that many libraries like Transformers and llama.cpp handle the complexities of the chat template for you.

### 3. Multi-Turn Conversations

- **No Thinking Content in History**: In multi-turn conversations, the historical model output should only include the final response. Thoughts from previous model turns must *not be added* before the next user turn begins, with the exception of tool call turns where thinking content should be preserved.

### 4. Modality order

For optimal performance with multimodal inputs, place:

- Image content **before** the text in your prompt.
- Audio content **after** the text in your prompt.

### 5. Variable Image Resolution

Aside from variable aspect ratios, Gemma 4 supports

variable image resolution through a configurable visual token budget, which controls how many tokens are used to represent an image. A higher token budget preserves more visual detail at the cost of additional compute, while a lower budget enables faster inference for tasks that don't require fine-grained understanding.

- The supported token budgets are: **70**, **140**, **280**, **560**, and **1120**.
    - Use *lower budgets* for classification, captioning, or video understanding, where faster inference and processing many frames outweigh fine-grained detail.
    - Use *higher budgets* for tasks like OCR, document parsing, or reading small text.

#### 6. Audio

Use the following prompt structures for audio processing:

- **Audio Speech Recognition (ASR)**

```
Transcribe the following speech segment in

Follow these specific instructions for for
* Only output the transcription, with no n
* When transcribing numbers, write the dig
```

- **Automatic Speech Translation (AST)**

```
Transcribe the following speech segment in
When formatting the answer, first output t
```

#### 7. Audio and Video Length

All models support image inputs and can process videos as frames whereas the E2B, E4B, and 12B models also support audio inputs. Audio supports a maximum length of 30 seconds. Video supports a maximum of 60 seconds assuming the images are processed at one frame per second.

### Model Data

Data used for model training and how the data was processed.

#### Training Dataset

Our pre-training dataset is a large-scale, diverse collection of data encompassing a wide range of

domains and modalities, which includes web documents, code, images, audio, with a cutoff date of January 2025. Here are the key components:

- **Web Documents**: A diverse collection of web text ensures the model is exposed to a broad range of linguistic styles, topics, and vocabulary. The training dataset includes content in over 140 languages.
- **Code**: Exposing the model to code helps it to learn the syntax and patterns of programming languages, which improves its ability to generate code and understand code-related questions.
- **Mathematics**: Training on mathematical text helps the model learn logical reasoning, symbolic representation, and to address mathematical queries.
- **Images**: A wide range of images enables the model to perform image analysis and visual data extraction tasks.